

Pulumi vs. CDK vs. Terraform: IaC comparison

I've interacted with Pulumi through a IaC comparison for a customer use case at AWS prior to this assignment. I primarily used CDK and CloudFormation at AWS and have used Terraform through a workshop.

Pulumi

Easier:

- Use real programming languages with full language features, leading to better debugging experience with standard language tools and IDEs
 - AI coding companions are generally very good at understanding a Pulumi program, which was so interesting
- Strong typing helps catch errors before deployment. Compile time & knowing what will happen before “up”ing out lowers risk
- Very straightforward with object-oriented approach. Coming from software engineering, barrier to entry is low.
- Consistent patterns and logic can be applied between all cloud providers create uniform experience.

Harder:

- I can see a steeper initial learning curve for infrastructure engineers without coding experience
- Compared to Terraform, it is a newer ecosystem. Fewer examples and community solutions/answers are available
- Package versioning can introduce complexity and compatibility issues – had to spend time as example code vs using a newer version had compatibility issues

AWS CDK (& Cloudformation)

Easier:

- Native to AWS. No state management is “needed” as AWS handles it in the console
- Forces to think AWS best practices – native IAM integration for deployments
- CDK: Constructs library provides high-level components that abstract complex patterns
- CDK: TypeScript/JavaScript support is excellent
- CFN: Drift detection and change management exists
- CFN: for simple deployments, easy to pick up “scripting”

Harder:

- No multicloud support
- As deployment gets complex, the abstraction layers and all the constructs make it harder to read. CFN files gets massively long and complex as infra complexity rises
- CDK is essentially a way to generate CloudFormation, which the generated CFN can be difficult to troubleshoot. CFN in general are very hard to troubleshoot and debug
- There is no “versioning” built in outside of CDK/CFN following CICD practices. Change management exists but is not the best.
- Managing multiple environments get complex
- CDK has a learning curve when moving beyond constructs. If not built properly from the ground up, changing later on how infra is provisioned can be troublesome

Terraform

Easier:

- Simple declarative syntax (HCL) is easier to learn initially. Very friendly towards script/config and traditional “infra engineers”.
- Massive ecosystem with modules and examples for almost everything. AWS has lots of examples, workshops and extensive support system. Documentation is also extensive.
- Provider-agnostic with consistent workflow across cloud providers

- State management is straightforward and explicit

Harder:

- Limited programming constructs (loops, conditionals) with awkward syntax. This makes complex logic to external tools or workarounds
- Conceptual understanding of state management and execution plan is critical – onboarding to an existing HCL infra could be harder
- Scaling complexity in large projects without proper module organization. File organization is critical, and that is not code-level. Operationally could cost more time
- Debugging can be challenging with error messages that lack context

Summary

- **Terraform:** Has wide range of examples to learn from and easy to write initially but can become complex as requirements grow; best for infrastructure engineers who prefer declarative syntax
- **Pulumi:** Most intuitive conceptual model for software developers; slightly steeper initial curve but better long-term maintainability
- **CDK:** Excellent for AWS-focused teams but can become unexpectedly complex; native AWS integration is powerful but limiting for multi-cloud
- **CFN:** Most tightly integrated with AWS but least flexible. Good for small teams on AWS and non-complex systems